

11강

spring Boot

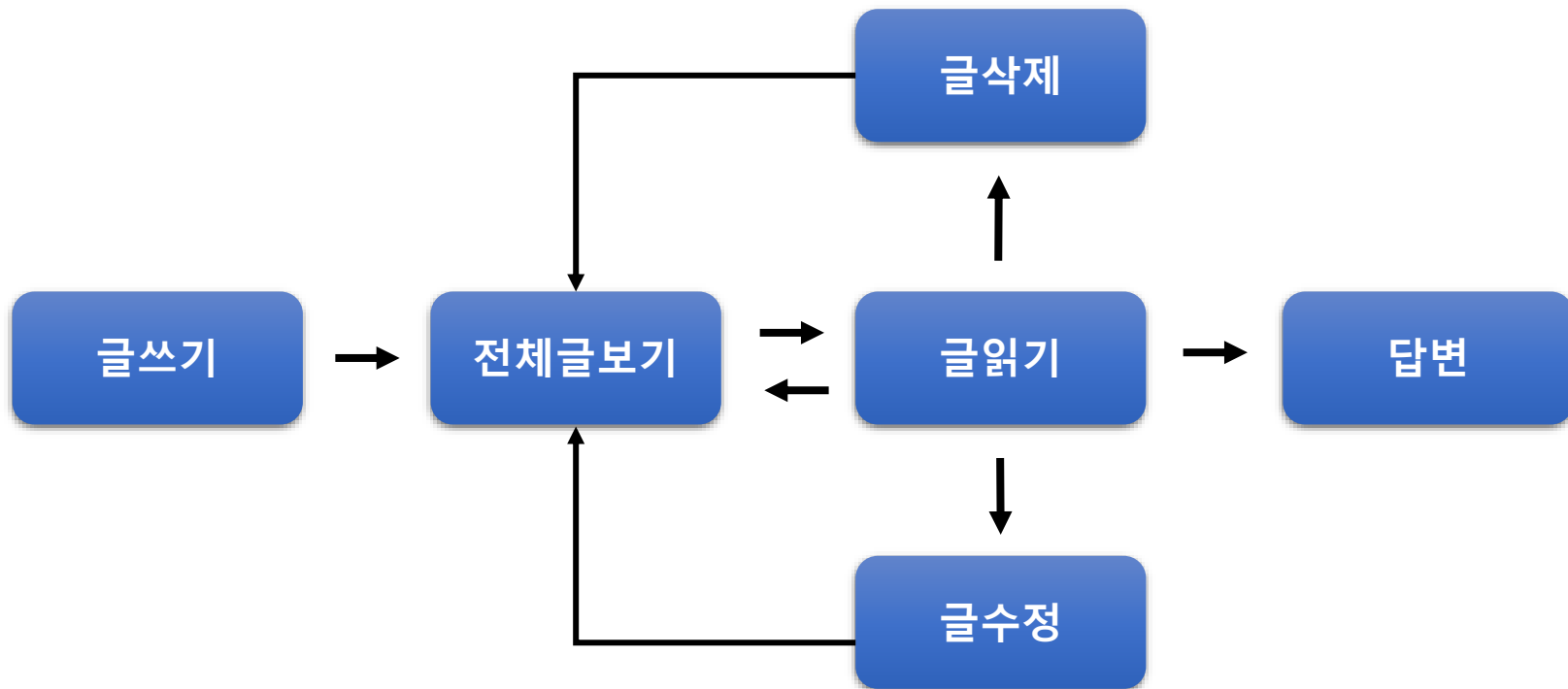
MVC 게시판





spring

➤ 게시판 구조





spring

➤ 게시판 DB 구조

테이블 구조

컬럼명	설명	비고
Num	글번호	자동증가 Int
Writer	글쓴이	String
Subject	글제목	String
writerPw	글 비밀번호	String
Reg_date	글 작성날짜	Date
Readcount	조회수	Int
Content	글내용	String



spring

➤ 게시판 DB 구조

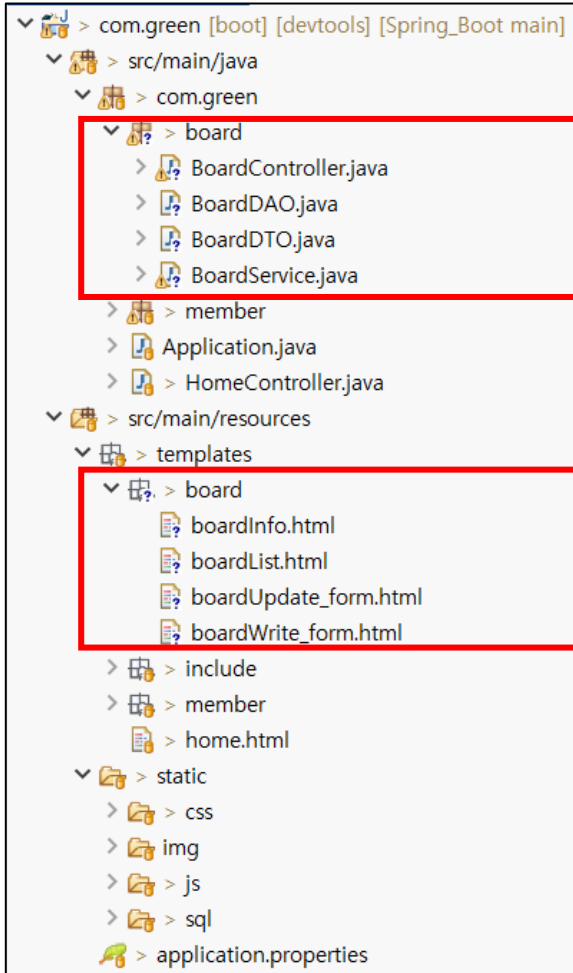
MySQL 테이블

```
-- board
create table board(
num int auto_increment primary key,
writer varchar(20),
subject varchar(30),
writerPw varchar(20),
reg_date datetime default now(),
readcount int default 0,
content varchar(1000)
);
```



spring

예제] MVC 구조를 이용하여 게시판을 작성한다.



1. 기존의 작성한 프로젝트를 이용하여 application.properties, build.gradle 파일들의 환경을 설정한다.
2. 환경설정이 끝나면 반드시 프로젝트 이름 오른쪽 버튼 -> Gradle -> Refresh gradle Project 를 실행한다.
3. src/main/resource -> board 패키지 생성 -> BoardController.java, BoardDAO.java, BoardDTO.java, BoardService.java 파일들을 각각 생성한다.
4. src/main/resource -> board 폴더 생성 -> boardWrite_form.html, boardList.html, boardInfo.html, boardUpdate_form.html 파일을 각각 생성한다.
5. src/main/resource -> static -> css -> board.css 파일을 생성한다.



spring

예제] com.green패키지 안에 -> src/main/java -> board -> BoardDTO 클래스를 생성하고 아래와 같이 작성한다. (반드시, getter, setter한다.)

```
package com.green.board;

public class BoardDTO {
    private int num; //글번호
    private String writer; // 글쓴이
    private String subject; // 글제목
    private String writerPw; // 글 비밀번호
    private String reg_date; // 글 작성일자
    private int readcount; // 조회수
    private String content; // 글 내용

    public int getNum() {
        return num;
    }
    public void setNum(int num) {
        this.num = num;
    }
    public String getWriter() {
        return writer;
    }
}
```

MVC 게시판

글 작성 페이지

(boardWrite_form)





spring

예제] com.green패키지 안에 -> src/main/resources -> board -> boardWrite_form.html 파일을 생성하고 아래와 같이 html을 작성한다.

```
<html lang="ko" xmlns:th="http://www.thymeleaf.com">
<head>
<meta charset="UTF-8">
<title th:replace="~/include/title.html::title">Member Join</title>
<link rel="stylesheet" th:href="@{/css/common.css}">
<link rel="stylesheet" th:href="@{/css/header_nav_footer.css}">
<link rel="stylesheet" th:href="@{/css/home.css}">
<link rel="stylesheet" th:href="@{/css/board.css}">
<script th:src="@{/js/member.js}"></script>
</head>
<body>
<header th:replace="~/include/header_nav_footer.html::header"></header>
<nav th:replace="~/include/header_nav_footer.html::nav"></nav>
<section>
<div id="section_wrap">
<div class="word">게시글 쓰기</div>
<div class="content">
<form th:action="@{/board/writePro}" method="POST" name="board_form">
<table width="600" border="1">
<tr height="40">
<td align="center" width="150">작성자</td>
<td width="450"><input type="text" name="writer"></td>
</tr>
<tr height="40">
<td align="center" width="150">제목</td>
<td width="450"><input type="text" name="subject" class="b_subject"></td>
</tr>
<tr height="40">
<td align="center" width="150">비밀번호</td>
<td width="450"><input type="password" name="writerPw"></td>
</tr>
</table>
</div>
</div>
</section>
</body>
</html>
```

← 필수 추가

[illegible]

➤ boardWrite_form.html 출력화면

게시글 쓰기

작성자	
제목	
비밀번호	
글내용	
글쓰기 다시작성 목록보기	



spring

- boardWrite_form.html에서 => [글쓰기] 버튼 클릭하면 글 작성
- 글쓰기에 필요한 DAO

```
@Autowired
private DataSource dataSource;

//하나의 게시글을 추가하는 메소드-----
public void insertBoard(BoardDTO bdto) {
    System.out.println("BoardDAO insertBoard()메소드 확인");

    String sql = "INSERT INTO board(writer,subject,writerPw,content) VALUES(?,?,?,?)";
    try{
        Connection conn = dataSource.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql);
        ){
            pstmt.setString(1, bdto.getWriter());
            pstmt.setString(2, bdto.getSubject());
            pstmt.setString(3, bdto.getWriterPw());
            pstmt.setString(4, bdto.getContent());
            pstmt.executeUpdate();
        }catch(Exception e) {
            e.printStackTrace();
        }
    }
}
```



spring

- boardWrite_form.html에서 => [글쓰기] 버튼 클릭하면 글 작성
- 글쓰기에 필요한 Service와 Controller

```
@Autowired
BoardDAO boardDao;

//하나의 게시글 추가하는 메소드
public void addBoard(BoardDTO bdto) {
    boardDao.insertBoard(bdto);
}
```

- 사용자가 글을 입력할 수 있는 빈 종이(boardWrite_form.html)를 화면에 띄워주는 컨트롤러 이다.

```
@Controller
public class BoardController {

    @Autowired
    BoardService boardService;

    // 1. 게시글 작성 폼
    @GetMapping("/board/write")
    public String boardWriteForm() {
        System.out.println("BoardController boardWriteForm() 호출");
        String nextPage = "board/boardWrite_form";
        return nextPage;
    }
}
```



spring

- boardWrite_form.html에서 => [글쓰기] 버튼 클릭하면 글 작성
- 글쓰기에 필요한 Service와 Controller
- 사용자가 입력한 제목, 내용 등의 데이터를 실제 데이터베이스(DB)에 영구적으로 저장하는 컨트롤러이다.

```
// 2. 게시글 저장 처리 (신규 추가 중요!)
@PostMapping("/board/writePro")
public String boardWritePro(BoardDTO bdto) {
    System.out.println("BoardController boardWritePro() 호출");

    // 서비스의 addBoard 호출하여 DB 저장
    boardService.addBoard(bdto);

    // 저장 후에는 '목록' 페이지로 이동 (Redirect)
    return "redirect:/board/list";
}
```


MVC 게시판

전체 글 목록페이지

(boardList)





spring

예제] com.green패키지 안에 -> src/main/resources -> board -> boardList.html 파일을 생성하고 아래와 같이 html을 작성한다.

```
<header th:replace=~{/include/header_nav_footer.html::header}></header>
<nav th:replace=~{/include/header_nav_footer.html::nav}></nav>
<section>
    <div id="section_wrap">
        <div class="word">게시글 보기</div>
        <div class="content">
            <table width="700" border="1">
                <tr height="40">
                    <td colspan="5" align="right">
                        <a th:href="@{/board/write}" class="write-btn">글쓰기</a>
                    </td>
                </tr>
                <tr height="40">
                    <td width="50" align="center">번호</td>
                    <td width="320" align="center">제목</td>
                    <td width="100" align="center">작성자</td>
                    <td width="150" align="center">작성일자</td>
                    <td width="80" align="center">조회수</td>
                </tr>
            </table>
        </div>
    </div>
</section>
```




spring

```
<tr th:each="li: ${list}">
    <td th:text="${li.num}"></td>
    <td>
        <a th:href="@{/board/boardInfo(num=${li.num})}" th:text="${li.subject}"></a>
    </td>
    <td th:text="${li.writer}"></td>
    <td th:text="${li.reg_date}"></td>
    <td th:text="${li.readcount}"></td>
</tr>
</table>
</div>
<div id="search_wrap" style="text-align: center; margin-top: 20px;">
    <form th:action="@{/board/list}" method="get">
        <select name="searchType" style="padding: 5px;">
            <option value="subject">제목</option>
            <option value="content">내용</option>
        </select>

        <input type="text" name="searchWord" placeholder="검색어를 입력하세요">

        <input type="submit" value="검색" class="searchBtn" >

        <input type="button" value="전체보기" class="searchBtn"
            th:onclick="'location.href=@{/board/list}'" >
    </form>
</div>
</div>
</section>
```



spring

➤ boardList.html 출력화면

전체 게시글 목록

글쓰기				
번호	제목	작성자	작성일자	조회수
3	어려워	달래	2026-01-28 08:57:52	0
2	재미있어	길동	2026-01-28 08:57:33	0
1	출다~	오티	2026-01-28 08:46:43	1

제목 ▼

검색어를 입력하세요

검색

전체보기



spring

- boardList.html 에서 => 전체 게시글 목록에 필요한 DAO
- 여러 개의 게시글을 출력하여야 하기때문에 ArrayList<E> 컬렉션을 이용해 작성한다.

```
// 전체 게시글을 검색하는 메소드-----
public List<BoardDTO> getAllBoard(){

    List<BoardDTO> boardList = new ArrayList<BoardDTO>();
    String sql = "SELECT * FROM board ORDER BY num DESC";
    try(
        Connection conn = dataSource.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql);
    ){
        ResultSet rs = pstmt.executeQuery();
        while(rs.next()) {
            BoardDTO bdto = new BoardDTO();
            bdto.setNum(rs.getInt("num"));
            bdto.setWriter(rs.getString("writer"));
            bdto.setSubject(rs.getString("subject"));
            bdto.setWriterPw(rs.getString("writerPw"));
            bdto.setReg_date(rs.getString("Reg_date").toString());
            bdto.setReadcount(rs.getInt("readcount"));
            bdto.setContent(rs.getString("content"));

            boardList.add(bdto);

        }
    }catch(Exception e) {
        e.printStackTrace();
    }
    return boardList;
}
```



spring

- boardList.html 필요한 Service와 Controller

```
//전체 게시글 출력하는 메소드
public List<BoardDTO> allBoard(){
    return boardDao.getAllBoard();
}
```

- DB에서 전체 게시글 목록을 가져와 모델(Model)에 담고, 목록 화면(boardList)으로 전달하는 역할을 담당하는 컨트롤러 이다.

```
// 3. 게시글 목록 보기 핸들러
@GetMapping("/board/list")
public String boardList(Model model) {
    System.out.println("BoardController boardList() 호출");

    List<BoardDTO> listboard = boardService.allBoard();
    model.addAttribute("list",listboard);
    String nextPage = "board/boardList";
    return nextPage;
}
```

MVC 게시판

게시글 상세 보기 페이지 (boardInfo)





spring

예제] com.green패키지 안에 -> src/main/resources -> board -> boardInfo.html 파일을 생성하고 아래와 같이 html을 작성한다.

```
<header th:replace=~{/include/header_nav_footer.html::header}></header>
<nav th:replace=~{/include/header_nav_footer.html::nav}></nav>
<section>
    <div id="section_wrap">
        <div class="word">게시글 보기</div>
        <div class="content">
            <table width="600" border="1">
                <tr height="40">
                    <td align="center" width="120">글번호</td>
                    <td align="center" width="180" th:text="${oneboard.num}"></td>
                    <td align="center" width="120">조회수</td>
                    <td align="center" width="180" th:text="${oneboard.readcount}"></td>
                </tr>
                <tr height="40">
                    <td align="center" width="120">작성자</td>
                    <td align="center" width="180" th:text="${oneboard.writer}"></td>
                    <td align="center" width="120">작성일</td>
                    <td align="center" width="180" th:text="${oneboard.reg_date}"></td>
                </tr>
                <tr height="40">
                    <td align="center" width="120">제목</td>
                    <td align="center" colspan="3" th:text="${oneboard.subject}"></td>
                </tr>
            </table>
        </div>
    </div>
</section>
```



spring

```
<tr height="80">
  <td align="center" width="120">글내용</td>
  <td align="center" colspan="3" th:text="${oneboard.content}"></td>
</tr>
<tr height="40">
  <td align="center" colspan="4">
    <input type="button" value="수정하기"
      th:onclick="|location.href='@{/board/update(num=${oneboard.num})}'|">

    <form th:action="@{/board/deletePro}" method="GET" style="display:inline;">
      <input type="hidden" name="num" th:value="${oneboard.num}">
      <input type="hidden" name="writerPw" th:value="${oneboard.writerPw}">

      <input type="submit" value="삭제하기" onclick="return confirm('정말 삭제하시겠습니까?');">
    </form>

    <input type="button" value="목록보기"
      th:onclick="|location.href='@{/board/list}'|">
  </td>
</tr>
</table>
</div>
</div>
</section>
```



spring

➤ boardInfo.html 출력화면

게시글 보기

글번호	3	조회수	1
작성자	달래	작성일	2026-01-28 08:57:52
제목	어려워		
글내용	어려워		
수정하기 삭제하기 목록보기			



spring

- boardInfo.html 에서 => 하나의 게시글 목록 상세보기에 필요한 DAO
- 하나의 게시글을 클릭할 때 마다 조회수가 1씩 증가되어야 하므로 , update, select쿼리를 같이 작성한다.

```
//하나의 게시글 검색 -----
// 하나의 게시글을 리턴받는 메소드 작성
public BoardDTO getOneBoard(int num) {

    BoardDTO bdto = new BoardDTO();
    String countsql = "UPDATE board SET readcount = readcount + 1 WHERE num=?";
    String sql = "SELECT * FROM board WHERE num=?";

    // Connection은 하나만 열어서 공유합니다.
    try (Connection conn = dataSource.getConnection()) {

        // 1. 조회수 증가 처리
        try (PreparedStatement pstmt = conn.prepareStatement(countsql)) {
            pstmt.setInt(1, num);
            pstmt.executeUpdate();
        }

        // 2. 게시글 정보 가져오기
        try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, num);
            ResultSet rs = pstmt.executeQuery();
            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    bdto.setNum(rs.getInt("num"));
                    bdto.setWriter(rs.getString("writer"));
                    bdto.setSubject(rs.getString("subject"));
                    bdto.setWriterPw(rs.getString("writerPw"));
                    bdto.setReg_date(rs.getString("Reg_date").toString());
                    bdto.setReadcount(rs.getInt("readcount"));
                    bdto.setContent(rs.getString("content"));
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return bdto;
}
```



spring

- boardInfo.html 필요한 Service와 Controller

```
//하나의 게시물 출력 메소드
public BoardDTO oneBoard(int num) {
    System.out.println("BoardService oneBoard()메소드 확인");
    return boardDao.getOneBoard(num);
}
```

- 글 번호(num)를 전달받아 해당 게시글의 상세 데이터를 DB에서 조회하고, 그 정보를 상세 보기 화면(boardInfo)으로 전달하는 역할을 하는 컨트롤러이다.

```
// 4. 하나의 게시물 상세 정보 보기 핸들러
@GetMapping("/board/boardInfo")
public String boardInfo(Model model, @RequestParam("num") int num) {
    System.out.println("BoardController boardInfo()메소드 호출"+ num);
    BoardDTO oneboardInfo = boardService.oneBoard(num);

    model.addAttribute("oneboard",oneboardInfo);
    String nextPage = "board/boardInfo";
    return nextPage;
}
```

MVC 게시판

게시글 수정 페이지

(boardUpdate_form)





spring

예제] com.green패키지 안에 -> src/main/resources -> board -> boardUpdate_form.html 파일을 생성하고 아래와 같이 html을 작성한다.

```
<header th:replace="~/include/header_nav_footer.html::header"></header>
<nav th:replace="~/include/header_nav_footer.html::nav"></nav>
<section>
  <div id="section_wrap">
    <div class="word">게시글 수정하기</div>
    <div class="content">
      <form th:action="@{/board/updatePro}" method="post">
        <input type="hidden" name="num" th:value="${oneboard.num}">

        <table width="600" border="1">
          <tr height="40">
            <td align="center" width="150">글번호</td>
            <td width="450" th:text="${oneboard.num}"></td>
          </tr>
          <tr height="40">
            <td align="center">작성자</td>
            <td th:text="${oneboard.writer}"></td>
          </tr>
          <tr height="40">
            <td align="center">제목</td>
            <td>
              <input type="text" name="subject"
                th:value="${oneboard.subject}" style="width:98%">
            </td>
          </tr>
        </table>
      </form>
    </div>
  </div>
</section>
```



spring

```
<tr height="150">
  <td align="center">글내용</td>
  <td><textarea name="content" rows="10" style="width:98%"
    th:text="${oneboard.content}"></textarea></td>
</tr>
<tr height="40">
  <td align="center" style="color: red;">비밀번호 확인</td>
  <td><input type="password" name="writerPw" required placeholder="비밀번호를 입력하세요"></td>
</tr>
<tr height="40">
  <td align="center" colspan="2">
    <input type="submit" value="수정완료">
    <input type="button" value="취소"
    th:onclick="|location.href='@{/board/boardInfo(num=${oneboard.num})}'|">
  </td>
</tr>
</table>
</form>
</div>
</div>
</section>
```



spring

➤ boardUpdate_form.html 출력화면

게시글 수정하기

글번호		3
작성자	달래	
제목	어려워	
글내용	어려워	
비밀번호 확인	비밀번호를 입력하세요	
수정완료		취소



spring

- boardUpdate_form.html 에서 => 게시글 수정에 필요한 DAO
- **게시글의 비밀번호와 번호가 같은 경우에만 수정을 진행한다.**

```
// 하나의 게시글 수정하는 메소드 -----
public int updateBoard(BoardDTO bdto) {
    System.out.println("BoardDAO updateBoard() 메소드 확인");

    int result = 0;
    // 제목과 내용만 수정 가능하도록 설정 (비밀번호 확인 로직은 보통 서비스나 쿼리 조건에 추가)
    String sql = "UPDATE board SET subject=?, content=? WHERE num=? AND writerPw=?";

    try (
        Connection conn = dataSource.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql);
    ) {
        pstmt.setString(1, bdto.getSubject());
        pstmt.setString(2, bdto.getContent());
        pstmt.setInt(3, bdto.getNum());
        pstmt.setString(4, bdto.getWriterPw());

        result = pstmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return result;
}
```



spring

- boardUpdate_form.html 필요한 Service와 Controller
- 전달받은 게시글 데이터를 DB에 업데이트하고, 수정 성공 여부(성공 시 true, 실패 시 false)를 판별하여 반환하는 역할을 담당하는 서비스 로직이다.

```
//하나의 게시글을 수정하는 메소드
public boolean modifyBoard(BoardDTO bdto) {
    System.out.println("BoardService modifyBoard()메소드 확인");

    int result = boardDao.updateBoard(bdto);

    if(result > 0) {
        System.out.println("게시글 수정 성공");
        return true;
    } else {
        System.out.println("게시글 수정 실패 (비밀번호 불일치 등)");
        return false;
    }
}
```




spring

- boardUpdate_form.html 필요한 Service와 Controller
- 글 번호(num)로 기존 내용을 조회하여 모델에 담고, 수정할 수 있는 입력 화면(boardUpdate_form)으로 전달하는 역할을 담당하는 컨트롤러 이다.

```
// 5. 게시글 수정 폼 이동 (기존 데이터를 가지고 이동)
@GetMapping("/board/update")
public String boardUpdateForm(@RequestParam("num") int num, Model model) {
    System.out.println("BoardController boardUpdateForm() 호출: " + num);

    // 기존 데이터를 불러와서 수정 폼에 채워주기 위해 상세정보 조회와 동일한 로직 사용
    BoardDTO oneboardInfo = boardService.oneBoard(num);
    model.addAttribute("oneboard", oneboardInfo);
    String nextPage = "board/boardUpdate_form";

    return nextPage; // 수정 폼 HTML 파일명
}
```



spring

- boardUpdate_form.html 필요한 Service와 Controller
- 수정된 데이터를 전달받아 DB 업데이트를 수행하고, 결과에 따라 목록 페이지(list)나 상세 페이지(boardInfo)로 이동시키는 역할을 담당하는 컨트롤러이다.

```
// 6. 게시글 수정 처리
@PostMapping("/board/updatePro")
public String boardUpdatePro(BoardDTO bdto, Model model) {
    System.out.println("BoardController boardUpdatePro() 호출");

    boolean isSuccess = boardService.modifyBoard(bdto);

    if(isSuccess) {
        // 수정 성공 시 목록으로 이동
        return "redirect:/board/list";
    } else {
        // 수정 실패 시 (비밀번호 틀림 등) 메시지 처리나 이전 페이지 유도가 필요할 수 있음
        // 여기서는 일단 다시 상세페이지로 리다이렉트
        return "redirect:/board/boardInfo?num=" + bdto.getNum();
    }
}
```

MVC 게시판

게시글 삭제 하기

(boardInfo)





spring

예제] com.green패키지 안에 -> src/main/resources -> board -> boardInfo.html 화면에서 [삭제하기] 버튼을 클릭하면 게시물 삭제를 진행한다.

게시글 보기

글번호	3	조회수	1
작성자	달래	작성일	2026-01-28 08:57:52
제목	어려워		
글내용	어려워		
수정하기 삭제하기 목록보기			



spring

- boardInfo.html 에서 => 하나의 게시글 삭제 필요한 DAO
- 하나의 게시글을 삭제할 때 비밀번호와 번호가 같아야 삭제를 진행한다.

```
// 하나의 게시글을 삭제하는 메소드 -----  
public int deleteBoard(int num, String writerPw) {  
    System.out.println("BoardDAO deleteBoard() 메소드 확인");  
  
    int result = 0;  
    // 번호와 비밀번호가 일치해야 삭제  
    String sql = "DELETE FROM board WHERE num=? AND writerPw=?";  
  
    try (  
        Connection conn = dataSource.getConnection();  
        PreparedStatement pstmt = conn.prepareStatement(sql);  
    ) {  
        pstmt.setInt(1, num);  
        pstmt.setString(2, writerPw);  
  
        result = pstmt.executeUpdate();  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
  
    return result;  
}
```



spring

- boardInfo.html 삭제에 필요한 Service와 Controller
- 글 번호(num)와 비밀번호(writerPw)를 대조하여 일치할 경우 DB에서 해당 게시글을 삭제하고, 그 성공 여부를 반환하는 역할을 담당하는 서비스 로직이다.

```
// 하나의 게시글을 삭제하는 메소드
public boolean removeBoard(int num, String writerPw) {
    System.out.println("BoardService removeBoard()메소드 확인");

    int result = boardDao.deleteBoard(num, writerPw);

    if(result > 0) {
        System.out.println("게시글 삭제 성공");
        return true;
    } else {
        System.out.println("게시글 삭제 실패 (비밀번호 불일치 등)");
        return false;
    }
}
```



spring

- boardInfo.html 삭제에 필요한 Service와 Controller
- 글 번호(num)를 전달받아 해당 게시글의 상세 데이터를 DB에서 조회하고, 그 정보를 상세 보기 화면(boardInfo)으로 전달하는 역할을 하는 컨트롤러이다.

```
// 7. 게시글 삭제 처리 (보통 삭제 전용 폼이나 상세페이지의 비밀번호 입력값 이용)
@GetMapping("/board/deletePro")
public String boardDeletePro(@RequestParam("num") int num,
                             @RequestParam("writerPw") String writerPw) {
    System.out.println("BoardController boardDeletePro() 호출");

    boolean isSuccess = boardService.removeBoard(num, writerPw);

    if(isSuccess) {
        return "redirect:/board/list";
    } else {
        // 삭제 실패 시 상세페이지로 복귀
        return "redirect:/board/boardInfo?num=" + num;
    }
}
```



- 전체적인 데이터 흐름 요약
- 사용자: boardWrite_form에서 글 작성 후 '글쓰기' 버튼 클릭.
- 브라우저: POST 방식으로 /board/writePro 경로에 데이터를 전송.
- Controller: @PostMapping("/board/writePro")가 실행되며, 매개변수 BoardDTO에 폼 데이터가 자동으로 담김.
- Service & DAO: boardService.addBoard(bdto)를 거쳐 boardDao.insertBoard(bdto)가 실행되어 실제 DB에 INSERT 쿼리가 날아감.
- Redirect: 저장이 완료되면 return "redirect:/board/list";를 통해 게시글 목록 화면으로 브라우저를 이동시킴